AI infrastructure in production

GPU scheduling and sharing on Kubernetes, vector databases, RAG pipelines, LLM gateways, LLM observability and evaluation, and cost and capacity for inference.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

Contents

1. GPU scheduling and sharing on Kubernetes (13 min)
2. Vector databases and retrieval operations (12 min)
3. RAG pipelines in production (12 min)
4. LLM gateways: routing, caching and rate limits (12 min)
5. LLM observability, evaluation and guardrails (12 min)
6. Cost and capacity engineering for inference (13 min)
Cheat sheet

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 1. GPU scheduling and sharing on Kubernetes

13 minutes. Expose GPUs to pods with the device plugin, share them with time slicing, MIG or MPS, and place workloads with taints, topology and queueing.

GPUs are scarce and expensive, and a default pod gets a whole one even if it uses a tenth. Good scheduling and sharing decide how much useful work each GPU hour buys.

What you will learn

- The device plugin and extended resources
- Whole GPU, time slicing, MIG and MPS
- Taints, node selectors and topology
- Queueing and gang scheduling for training
- Monitoring utilisation

Device plugin

The NVIDIA device plugin advertises `nvidia.com/gpu` as an extended resource. A pod asks for it in limits and the scheduler places it on a node with free GPUs. GPUs are not overcommitted by default, so one pod requesting one GPU holds the whole device.

Sharing

Time slicing lets several pods share a GPU in turns with no memory isolation, which suits light inference and development. MIG on supported cards partitions a GPU into isolated slices with their own memory and compute. MPS lets processes share compute concurrently. Choose by isolation needs and by what your cards support.

Placement

Taint GPU nodes so that only pods that tolerate the taint land on them, and use node selectors or affinity for a card type. For multi GPU jobs, topology matters: GPUs connected by a fast interconnect on the same node perform much better than GPUs across nodes.

Queues and gangs

A distributed training job needs all its workers at once. Without gang scheduling, half the workers start and wait while holding GPUs. Queue systems such as Kueue or Volcano admit a job only when all resources are available and share quota between teams.

Monitoring

Export DCGM metrics for GPU utilisation, memory and power. A GPU that is allocated but idle is the most common waste. Alert on allocation with low utilisation over hours and report it per team.

Diagram: GPU scheduling and sharing on Kubernetes

Part	What it does	What goes wrong
Pod requesting GPU	Asks for devices.	Whole GPU for little work.
Scheduler and queue	Places and admits.	Partial gang.
GPU node	Device plugin and slices.	Idle device.
DCGM metrics	Utilisation.	Not watched.

Flow: Pod requesting GPU to Scheduler and queue / Scheduler and queue to GPU node / GPU node to DCGM metrics

MARGA by EknathaLabs marga.eknathalabs.com

Incident: GPUs allocated and idle

Incident: *Eight GPUs were reserved for a notebook and used less than five percent for a week.*

```
$ kubectl exec dcgm-exporter-x -- curl -s localhost:9400/metrics | grep DCGM_FI_DEV_GPU_UTIL | hea \
    d -2
DCGM_FI_DEV_GPU_UTIL{gpu="0",pod="nb-ana"} 3
DCGM_FI_DEV_GPU_UTIL{gpu="1",pod="nb-ana"} 0
(this is a simulated illustration, not a real incident)
```

The notebook held its GPUs between experiments. The team moved light work to time sliced GPUs, added an idle culler that releases notebooks after two hours and published per team utilisation so owners could see the waste.

Common mistakes

- Giving whole GPUs to light workloads
- No taints so CPU pods land on GPU nodes
- Starting partial training jobs without gang scheduling
- Mixing isolation levels without understanding them
- Never watching utilisation

Check yourself

How does a pod request a GPU?

Answer: As an extended resource in limits

What does MIG provide?

Answer: Isolated GPU slices with their own memory

Why use gang scheduling?

Answer: All workers of a training job must start together

What is a common GPU waste?

Answer: Allocated but idle devices

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Vector databases and retrieval operations

12 minutes. Operate a vector database: choose an index type, size memory, plan updates and backups, and measure recall and latency.

Retrieval systems find text by meaning using vectors. Operating one is a database problem with new trade-offs between speed, memory and accuracy.

What you will learn

- Embeddings and similarity search
- Index types: flat, IVF and HNSW
- Sizing memory and disk
- Filtering, updates and consistency
- Measuring recall, latency and cost

Embeddings

An embedding model turns text into a vector of a few hundred to a few thousand numbers. Texts with similar meaning have vectors that are close under a measure such as cosine similarity. Changing the embedding model changes every vector, so all stored data must be re-embedded.

Index types

A flat index compares the query to every vector and is exact but slow at scale. Approximate indexes such as HNSW graphs or IVF clusters trade a little accuracy for much lower latency. HNSW is fast and memory heavy, IVF uses less memory and needs training on your data.

Sizing

A vector of 1536 floats takes about six kilobytes, so ten million vectors need roughly sixty gigabytes before the index overhead. Quantisation reduces size at some accuracy cost. Keep hot indexes in memory and plan for replicas for availability and read load.

Filters and updates

Most queries filter by metadata, such as tenant or date. Filtering after the search can return too few results, and filtering before can slow the search, so test your engine with realistic filters. Frequent updates and deletes may need background compaction or index rebuilds.

Measuring

Recall is the share of the true nearest neighbours that the index returns. Measure it against an exact search on a sample, together with latency at the 95th and 99th percentile. Tune index parameters to meet a recall target at the lowest cost, and re-test after growth.

Diagram: Vector databases and retrieval operations

Part	What it does	What goes wrong
Embedding model	Text to vector.	Model change.
Vector index	Approximate search.	Low recall.
Metadata filters	Tenant and date.	Few results.
Application	Uses top results.	Slow queries.

Flow: Embedding model to Vector index / Vector index to Metadata filters / Metadata filters to Application

MARGA by EknathaLabs marga.eknathalabs.com

Incident: Search quality dropped after an embedding upgrade

Incident: *Retrieval started returning unrelated documents after a model version change.*

```
$ psql -c "select count(*) from docs where embed_model = 'v2'"
 count
-------
 41000
$ psql -c "select count(*) from docs where embed_model = 'v1'"
 count
-------
 960000
(this is a simulated illustration, not a real incident)
```

New documents were embedded with the new model while old ones kept the old vectors, so queries compared vectors from different spaces. The team re-embedded the whole collection into a new index, switched over at once and stored the model version with every vector to catch this in future.

Common mistakes

- Mixing vectors from different models in one index
- Indexes sized without overhead
- Ignoring recall and only watching latency
- Filtering that returns too few results
- No backups of the source documents

Check yourself

What does an approximate index trade?

Answer: A little accuracy for speed

What happens when the embedding model changes?

Answer: All vectors must be recomputed

What is recall?

Answer: Share of true nearest neighbours returned

What drives memory need?

Answer: Vector count, dimensions and index overhead

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. RAG pipelines in production

12 minutes. Build a retrieval augmented generation pipeline with ingestion, chunking, retrieval, reranking and prompts, and make it observable and safe to update.

Retrieval augmented generation gives a model your documents at question time. Most failures come from the pipeline around the model: stale data, poor chunks and no way to tell what went wrong.

What you will learn

- The pipeline stages
- Ingestion and chunking choices
- Retrieval, hybrid search and reranking
- Prompt construction and citations
- Freshness, access control and updates

Stages

Documents are ingested, split into chunks, embedded and indexed. At question time the system retrieves candidate chunks, optionally reranks them, builds a prompt and calls the model, then returns the answer with sources. Each stage can fail separately, so measure each one.

Chunking

Chunks that are too small lose context and chunks that are too large dilute the match and use prompt space. A few hundred tokens with some overlap and boundaries at headings is a common start. Keep metadata such as the title, section and source link with each chunk.

Retrieval

Combine vector search with keyword search, since exact names and error codes are matched better by keywords. A reranker model reorders the top candidates by relevance and often improves answer quality more than a larger model. Retrieve more than you send and keep only the best few.

Prompts and citations

Tell the model to answer only from the supplied text and to say when the answer is not there. Return the sources so users can check. Treat retrieved text as untrusted: a document can contain instructions that try to change the model behaviour.

Freshness and access

Track document versions and re-index changes on a schedule or by event. Enforce access control at retrieval time with metadata filters, so a user only gets chunks they may read. Test with questions that must be answered from recent or restricted documents.

Diagram: RAG pipelines in production

Part	What it does	What goes wrong
Ingestion and chunking	Prepares documents.	Poor chunks.
Retrieval and rerank	Finds context.	Wrong passages.
Model call	Generates answer.	Ignores context.
Evaluation	Measures quality.	Not measured.

Flow: Ingestion and chunking to Retrieval and rerank / Retrieval and rerank to Model call / Model call to Evaluation

MARGA by EknathaLabs marga.eknathalabs.com

Incident: A confident answer from a deleted policy

Incident: *The assistant quoted a policy that had been withdrawn a month earlier.*

```
$ psql -c "select doc_id, updated_at, deleted from chunks where doc_id = 'policy-17' limit 2"
 doc_id    | updated_at | deleted
 policy-17 | 2026-08-02 | f
 policy-17 | 2026-08-02 | f
(this is a simulated illustration, not a real incident)
```

The source system marked the document deleted, but the index never received the delete event. The ingestion job was changed to process deletes, a nightly reconciliation compares the index with the source and answers now show the document date.

Common mistakes

- No handling of deleted or updated documents
- Chunks without source metadata
- Skipping evaluation and judging by a few demos
- Ignoring access control at retrieval
- Trusting retrieved text as safe instructions

Check yourself

What does retrieval add to a model call?

Answer: Relevant context from your data

Why combine keyword and vector search?

Answer: Keywords match exact names and codes better

How should access control be applied?

Answer: At retrieval with metadata filters

What keeps answers fresh?

Answer: Processing updates and deletes from the source

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. LLM gateways: routing, caching and rate limits

12 minutes. Put a gateway in front of model providers for routing, fallbacks, caching, quotas and key management, and understand its failure modes.

When many teams call model APIs directly, keys leak, costs surprise and outages spread. A gateway gives one place for policy, visibility and resilience.

What you will learn

- What a gateway does
- Routing and fallback between models and providers
- Caching: exact and semantic
- Rate limits, quotas and budgets
- Keys, logging and privacy

Functions

A gateway exposes one API to applications and forwards to providers or self hosted models. It holds the provider keys, authenticates callers, records usage per team and applies limits. Applications switch models by changing a name in configuration, not code.

Routing and fallback

Route simple requests to cheaper, faster models and hard ones to stronger models. Fall back to a second provider on errors or timeouts, with a retry budget so retries do not multiply load. Test that fallback models give acceptable answers, because behaviour differs.

Caching

An exact cache returns a stored answer for an identical request, which is safe for deterministic calls. A semantic cache returns an answer for a similar request, which saves more but can return a wrong answer for a question that differs in a small, important detail. Use it only where mistakes are cheap and set a similarity threshold carefully.

Limits and budgets

Limit requests and tokens per minute per team and per key, and set monthly budgets with alerts. Limit the maximum output length to stop runaway generations. Return clear errors with a retry time so clients can back off.

Keys, logs and privacy

Never give provider keys to applications. Log metadata such as model, tokens, latency and cost for every call. Logging prompts and outputs is useful for debugging but may contain personal data, so redact it, restrict access and set a short retention period.

Diagram: LLM gateways: routing, caching and rate limits

Part	What it does	What goes wrong
Applications	Call one API.	Direct keys.
Gateway	Auth, limits, routing.	Single point of failure.
Cache	Saves calls.	Wrong semantic hit.
Model providers	Generate text.	Outage.

Flow: Applications to Gateway / Gateway to Cache / Gateway to Model providers

MARGA by EknathaLabs
marga.eknathalabs.com

Incident: A retry storm during a provider outage

> **Incident:** *When a provider slowed down, retries from every service tripled the load and the gateway fell over.*

```
$ curl -s localhost:4000/metrics | grep -E 'requests_total|retries_total' | head -3
gateway_requests_total{status="429"} 18420
gateway_retries_total 52211
(this is a simulated illustration, not a real incident)
```

Each client retried three times and the gateway retried again, multiplying traffic. A shared retry budget, exponential backoff with jitter, a circuit breaker that failed fast to the fallback provider and per team limits stopped the next incident from spreading.

Common mistakes

- Provider keys inside applications
- Fallback to a model nobody tested
- Semantic cache for exact answers such as prices
- Unlimited output length
- Logging full prompts with personal data forever

Check yourself

Why use a gateway?

Answer: One place for keys, limits, routing and usage

What is a risk of semantic caching?

Answer: Returning an answer for a similar but different question

What should limits include?

Answer: Requests, tokens and budgets per team

How do you avoid retry storms?

Answer: Retry budgets, backoff with jitter and circuit breakers

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. LLM observability, evaluation and guardrails

12 minutes. Trace LLM calls, evaluate quality with test sets and graders, add input and output guardrails and watch for drift in cost and quality.

A model that is up can still be wrong. Observability for LLM systems must cover quality, safety and cost, not only latency and errors.

What you will learn

- What to log and trace for each call
- Offline evaluation with test sets
- Online signals and human feedback
- Guardrails: input, output and tools
- Drift: model, data and prompt changes

Tracing

Record each step as a span: retrieval, prompt, model call and tool calls. Capture model name, token counts, latency, cost and a prompt version. A single trace should show which context was retrieved and what the model returned, so you can see why an answer was wrong.

Offline evaluation

Build a test set of real questions with expected answers or checks. Score with exact checks where possible, and with a grader model plus periodic human review for open answers. Run the set on every change to prompt, model or retrieval and block releases that regress.

Online signals

Track thumbs up and down, retries, answer abandonment and escalations to humans. Sample conversations for review. Combine these with cost and latency per request to find the changes that matter.

Guardrails

Check inputs for prompt injection and sensitive data, and outputs for policy violations, personal data and format errors. Limit what tools a model can call and require confirmation for risky actions. Guardrails reduce risk but are not perfect, so design for the case where one fails.

Drift

Providers update models, documents change and users ask new things. Pin model versions where possible, keep prompts in version control, rerun the test set on a schedule and alert on shifts in refusal rate, length or cost.

Diagram: LLM observability, evaluation and guardrails

Part	What it does	What goes wrong
LLM call	Traced with metadata.	No trace.
Test set and graders	Block regressions.	Stale tests.
Guardrails	Filter input and output.	False negatives.
Dashboards and alerts	Quality, cost, latency.	Only latency.

Flow: LLM call to Test set and graders / LLM call to Guardrails / Guardrails to Dashboards and alerts

MARGA by EknathaLabs marga.eknathalabs.com

Incident: A silent change after a model update

Incident: *The provider updated a model alias and answers became longer and costlier overnight.*

```
$ curl -s localhost:9090/api/v1/query --data-urlencode 'query=avg(llm_output_tokens)' | head -c 14 \
    0
{"status":"success","data":{"result":[{"value":[1731700000,"412"]}]}}
$ curl -s localhost:9090/api/v1/query --data-urlencode 'query=avg(llm_output_tokens offset 1d)' |  \
    head -c 140
{"status":"success","data":{"result":[{"value":[1731613600,"236"]}]}}
(this is a simulated illustration, not a real incident)
```

The application used a moving alias, so behaviour changed without a deploy. The team pinned an explicit model version, added an alert on average output length and cost per request and scheduled a nightly run of the evaluation set.

Common mistakes

- Watching only latency and errors
- No test set, relying on demos
- Moving model aliases in production
- Guardrails as the only defence
- Prompts edited in a dashboard with no history

Check yourself

What should a trace of an LLM call include?

Answer: Context, prompt version, model, tokens and cost

When do you run the evaluation set?

Answer: On every change to prompt, model or retrieval

What causes drift?

Answer: Model updates, document changes and new user questions

What is true of guardrails?

Answer: They reduce risk but can fail

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Cost and capacity engineering for inference

13 minutes. Estimate inference cost per request, size capacity from tokens per second, use batching, quantisation and caching, and choose between hosted and self hosted models.

Inference cost grows with traffic and context length. A clear cost model lets you decide where to optimise and when running your own GPUs beats an API.

What you will learn

- Cost per request: tokens in, tokens out
- Capacity from throughput and latency targets
- Batching, quantisation and caching
- Autoscaling GPU workloads
- Hosted API versus self hosted

Cost per request

API prices are quoted per million input and output tokens. Output tokens usually cost several times more. Long prompts, large retrieved context and chat histories dominate cost, so trimming them is often the biggest saving.

Capacity

Throughput is measured in tokens per second per GPU, and it depends on the model size, precision, batch size and context length. Divide your peak tokens per second by the throughput of one GPU to estimate the count, add headroom for failures and test with realistic prompts, since benchmark numbers rarely match your traffic.

Optimisation

Continuous batching serves many requests together and raises utilisation. Quantisation to eight or four bits cuts memory and increases speed with a small quality loss that you must measure. Prefix caching reuses the computed prompt start, which helps when many requests share a long system prompt.

Autoscaling

GPU nodes take minutes to start and models take more minutes to load, so reactive scaling is slow. Scale on queue depth or tokens per second, keep a warm minimum, pre-pull images and weights and schedule scale ups before known peaks.

Hosted or self hosted

Hosted APIs have no fixed cost and no operations, and suit low or spiky traffic. Self hosting pays off at steady high volume or for data control, but you carry staff time, spare capacity and upgrades. Compare total cost per million tokens at your real utilisation.

Diagram: Cost and capacity engineering for inference

Part	What it does	What goes wrong
Requests	Tokens in and out.	Long context.
Batching and cache	Raises throughput.	Cold cache.
GPU replicas	Tokens per second.	Slow start.
Cost model	Per million tokens.	Low utilisation.

Flow: Requests to Batching and cache / Batching and cache to GPU replicas / GPU replicas to Cost model

MARGA by EknathaLabs
marga.eknathalabs.com

Incident: A self hosted model that cost more than the API

Incident: *A team bought GPUs to save money and found the cost per token was three times higher.*

```
$ curl -s localhost:9090/api/v1/query --data-urlencode 'query=avg(DCGM_FI_DEV_GPU_UTIL)' | head -c \
     150
{"status":"success","data":{"result":[{"value":[1731700000,"15"]}]}}
$ curl -s localhost:9090/api/v1/query --data-urlencode 'query=sum(increase(tokens_served_total[30d \
    ]))/1e6' | head -c 150
{"status":"success","data":{"result":[{"value":[1731700000,"980"]}]}}
(this is a simulated illustration, not a real incident)
```

Two GPUs at about three dollars an hour served 980 million tokens a month at fifteen percent utilisation, which cost more per million tokens than the hosted price. The team moved light traffic to the API, kept the GPUs for steady high volume jobs and raised utilisation with batching before buying more.

Common mistakes

- Benchmarking with unrealistic prompts
- No headroom for node failures
- Self hosting at low utilisation
- Ignoring context length in cost
- Scaling on CPU instead of queue depth

Check yourself

What dominates request cost?

Answer: Input and output tokens, especially long context

How do you estimate GPU count?

Answer: Peak tokens per second divided by one GPU throughput, plus headroom

What does continuous batching do?

Answer: Serves many requests together to raise utilisation

When does self hosting pay off?

Answer: High, steady volume or strict data control

MARGA by EknathaLabs
marga.eknathalabs.com

Cheat sheet: AI infrastructure

GPU sharing, RAG checks and gateway settings.

GPU sharing

Method	Isolation	Use
Whole GPU	Full	Training and large models
MIG	Hardware slices	Small, steady services
Time slicing	None	Light, bursty dev workloads

RAG checks

Check	Question
Freshness	When was the index last updated?
Recall	Do the right chunks appear in the top k?
Grounding	Does the answer cite retrieved text?

Gateway settings

Setting	Purpose
Rate limit	Protect quotas
Cache	Cut repeated cost
Fallback	Survive provider errors
Budget	Cap spend per team

MARGA by EknathaLabs
marga.eknathalabs.com